

Floyd-Warshall Algorithm — Project Report

I. Introduction

In this project, we implemented the Floyd-Warshall algorithm, which computes the shortest path between all pairs of vertices in a weighted directed graph. It can handle negative weights and detect absorbing circuits, which are negative cycles that make any shortest path undefined.

To make our work more concrete, we applied it to the currency exchange context.

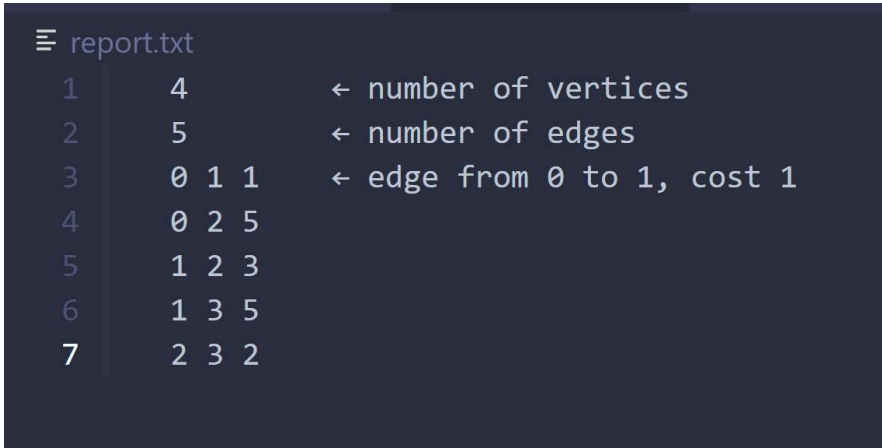
The goal is to test whether the Floyd-Warshall algorithm could be used to find interesting conversions between two currencies and maybe even find a way to perform several conversions to go back to the initial currency with more money.

II. The Representation of the problem

A graph is made of vertices (points) and edges (directed connections between points). Each edge has a weight, which in our context represents a cost or a rate.

In the currency exchange application, vertices are currencies, and edges are conversion rates. For example, an edge from vertex 0 to vertex 1 with weight 1.1585 means that 1 unit of currency 0 gives 1.1585 units of currency 1.

We store graphs in .txt files following this format :



```
≡ report.txt
1      4      ← number of vertices
2      5      ← number of edges
3      0 1 1   ← edge from 0 to 1, cost 1
4      0 2 5
5      1 2 3
6      1 3 5
7      2 3 2
```

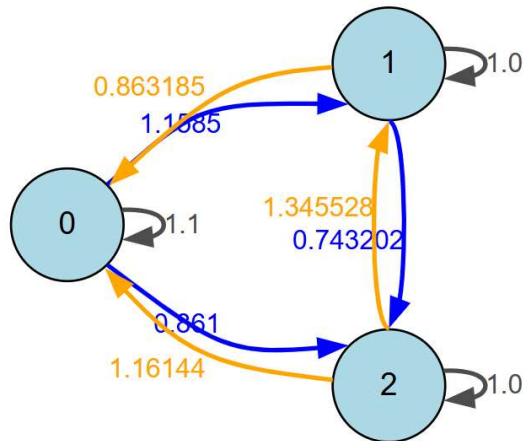
The program converts this into a Python dictionary where each vertex maps to its neighbors and their costs, which is then displayed as an adjacency matrix.

The currency-exchange can be modeled with a directed weighted graph such that :

- the vertices are the currency

- the weights are the exchange rate
- each arc from i to j is a conversion from currency i to j

To apply this in a real situation, we build a directed graph stored in the file graph_14.txt with 21 vertices and 441 arcs, allowing each currency to be converted into any other one. If we take only three vertices to illustrate the situation we get:



III. Floyd-Warshall Algorithm used for currency-exchange

We used our implementation of the Floyd-Warshall algorithm to find the shortest path between the pair of vertices (i,j). In the file floyd_algorithm, the function floyd_warshall_algorithm initialized:

- a matrix L to store the path costs
- a matrix P to store the predecessors
- the diagonal to 0
- the missing arc to 10^8 (to represent the infinity)

Then, for each intermediate vertices k, it checks if there is any better path to go from i to j. We keep the minimum between the previous path cost and the sum of the path cost through the intermediate vertices.

As for the data, we took mean values from 19th March of the 21st most popular currencies, leading to a grand total of 441 edges. They were gathered using an AI for simplicity's sake. The exchange rates have been stored in graph 14.

IV. Analysis of the result

The result of the algorithm of the graph_14 is stored in the file result_graph_14.txt, showing the shortest path for each vertex and the total cost of each path.

For example, the shortest path from 2 to 5 is the direct one with 1.81882.

	0	1	2	3	4	5
0	0	0	0	8	0	0
1	1	1	1	8	1	1
2	2	2	2	8	2	2

	0	1	2	3	4	5
0	0	1.1585	0.861	14.5909	0.958	1.566
1	0.863185	0	0.743202	14.1042	0.826931	1.35175
2	1.16144	1.34553	0	14.8746	1.11266	1.81882

With our model, the shortest path is frequently the direct one. This underlines a problem regarding the context where we try to apply the algorithm.

In the Floyd-Warshall algorithm, we compute the shortest additive path between i and j . Nevertheless, this is not coherent with the currency exchange since it works with multiplication. If we take the value from the graph of three vertices we can notice the following:

- In a financial point of view, to go from 0 to 2 passing through 1 we should do:
 $1.1585 * 0.743202 \approx 0.861$
- Instead, using Floyd-Warshall we do: $1.1585 + 0.743202 = 1.901702$

Here going from 0 to 2 has a smaller cost than the one using the intermediate vertex 1. But using the product seems to show that the shortest path is the same as the direct one.

We compute the shortest path on the graph rates, but not the most interesting exchange process. So, what is the path maximizing the final amount of money after conversions?

This is also connected to the notion of absorbing cycle. In Floyd-Warshall algorithm, this is a negative cycle leading to an undefined shortest path because the total cost decreases indefinitely. We check the values below 0 in the diagonal in the final matrix in the function `detect_absorbing_circuit`. In the context of currency exchange, it could allow to end the conversion with more money than at the beginning. Thus, the product of the exchange rate must be bigger than one.

To be able to find it, we could transform the values such that it fits with the Floyd-Warshall sum by replacing the rate with a negative logarithm. Then doing the sum of those values could be the same as having the logarithm of their product, and we could use the exponential to convert back the values. Therefore, we can keep the $-\log(\text{rate})$ to find the absorbing cycle using the Floyd Warshall algorithm. However, following those steps, we find absorbing cycles everywhere, as our values are not between 0 and 1, and

some of them go way over. Consequently, their logarithm is positive, and their opposite is negative, creating negative weights.

Lastly, rates are based on the same currency as reference, like USD, so, they are almost all compatible with other currencies. For instance, using the product, going from 0 to 2 should give the same as going from 0 to 2 passing through 1. And this is what we noticed, getting 0.861 for both paths.

Also, going from 0 to 1, 2 and then coming back to the initial vertex should give almost 1. We can also take the values from the same example as the previous one:

$$1.1585 \cdot 0.743202 \cdot 1.16144 \approx 0.9999$$

Thus, the data that we use in our application represents an equilibrated market: there is no opportunity to exploit.

V. Conclusion

All in all, the implementation of the Floyd-Warshall algorithm computes the path with the minimal values between two vertices and it can also detect the absorbing circuits. However, the currency exchange application shed light on logical issues. While Floyd-Warshall minimizes the sum of path values, the exchange rates must be computed using multiplication. Because of that, the shortest path does not always indicate the most interesting way to convert money. To find a more suitable model, we tried to use the negative logarithm function, but it led to negative cycles detected as absorbing cycles. Even if we update our model, the dataset of rates is too balanced, avoiding profit opportunities.

It is important to mention that exchange rates vary each second. They are impacted by variations on the stock market, new policies, or new events. Hence, the only reliable and profitable method through exchanging money is to buy shares of a company in a currency and wait for impactful variations that will make either the action or the currency grow, since the share percent will be static. Some might consider it a gamble, others a skill. Maybe someone will one day find the secret formula to infinite money, but this person was definitely not Mr. Floyd and neither Mr. Warshall.